AdmitAssist: a triage and reply assistant for school admission inquiries

> AdmitAssist | MBA ZG583 Management of AI Products | GOLLAPUDI SIVA NAGA JNANA TEJA | 2025MB26205 | GitHub: jnanateja009

Team details

Item	Detail
Team name	TEAM_2025MB26205_AdmitAssist (solo)
Member	GOLLAPUDI SIVA NAGA JNANA TEJA
Roll number	2025MB26205
GitHub ID	jnanateja009
Topic approval	AssignmentBriefing issue #6 (approved, closed)
Option	C: improve a real workflow (my father's school admissions office)

I am doing this alone, so I play every role: product manager, designer, builder and tester. Section 12 explains who did what, including what AI tools did.

What AdmitAssist is, in one paragraph

During admission season a school's front office gets parent messages from WhatsApp, voice notes, phone calls, walk-ins, the website form and e-mail. One person reads every message, works out which class the parent wants and how serious they are, and types a reply from memory. Serious parents can wait behind casual ones, answers change depending on who replies, and nothing is recorded. AdmitAssist reads each message, pulls out the class, timeframe and phone number, gives a priority (HIGH, MEDIUM or LOW) with the reasons shown, sends sensitive messages (complaints, fee help, a child's health, safety) straight to a person with no AI draft, and writes a draft reply that uses only facts the school has approved. A person reads every draft and presses Send in WhatsApp. The tool cannot send anything.

My recommendation

Pilot. Run a 10-week assisted pilot at my father's school (21 Dec 2026 to 28 Feb 2027) after two gates: the live model must pass the launch checks on 100 real labelled messages (Gate A, 6 Dec 2026), and a two-week shadow run must meet the targets (Gate B, 20 Dec 2026). In parallel, sell to 3 schools at Rs 1,499 per admission-season month, first invoice on 1 Mar 2027. Details are in 01_Executive_Summary.md.

How to read this repository

#	File	What it answers
01	01_Executive_Summary.md	Problem, who it is for, why AI, the evidence so far, recommendation
02	02_Product_Strategy.md	Stakeholders and what each values, jobs to be done, opportunity matrix, build vs buy vs API, first wedge, course case (Yellow.ai)
03	03_Product_Requirements_Document.md	The build contract: definitions, requirements with acceptance tests, what is configurable vs fixed, MVP scope, prioritisation

#	File	What it answers
03b	03b_Build_Contract_Annex.md	What exactly gets built, exhibits with checksums, software acceptance vs AI gates, binding prompt templates, permissions, life cycle, failure behaviour. Wins over other documents.
04	04_Workflow_and_Trust_Design.md	Today's workflow and the V1 workflow as swimlanes, each AI step's reliability, screen, cost and human check, uncertainty and fallback, wireframes
05	05_Data_Model_and_Evaluation.md	The AI task, data, labelling, baseline, model choice, metrics with targets, results so far, launch thresholds
06	06_Business_and_Unit_Economics.md	Price, cost to serve, unit economics, stress test, competition
07	07_Roadmap.md	V1 plan with dates to first revenue, V2 and V3 sketches
08	08_Ethics_Governance_and_Risk.md	Harm scenarios, risk register, data protection (DPDP), launch blockers, governance
09	09_Prototype_Guide.md	How to run the working prototype and what it does and does not prove
10	10_AI_Collaboration_Log.md	AI tools used, the prompts, where AI was wrong, what I corrected
11	11_Self_Evaluation_Vendor_Readiness.md	Faculty instruction: gave the documents to an AI, asked "can you build this?" and for a self-evaluation; what I changed
12	12_Individual_Contribution_Statement.md	Who did what

Folder	Contents
`diagrams/`	All diagrams as PlantUML code (swimlanes, states, context, deployment, wireframes, plan). Paste any file into plantuml.com to see it.
`prototype/`	The working web app, the shared engine, the school settings file, 48 sample messages, recorded AI responses and the evaluation script
`business/`	`unit_economics.py`, which produces every money figure in 06 and the deck
`tools/`	`plantuml_links.py` (diagram links), `draw_swimlane_preview.py` (preview images for the PDFs), `wording_check.py` (vague-word and banned-word check)
`deck/`	5-slide pitch deck (PowerPoint and PDF)
`pdf/`	PDF copies of documents 01 to 12 for reading

Run the prototype in two minutes

```
cd prototype
python -m http.server 8000        # then open http://localhost:8000
```

Click **Load sample messages**. No API key is needed for the recorded demo. For live AI, open **AI connection**, choose Live AI and paste an OpenAI-compatible API key. Full steps: 09_Prototype_Guide.md.

Re-run the tests and evaluation (Node 18 or newer): `node prototype/eval/unit_tests.js` (51/51), `node prototype/eval/run_eval.js --mode baseline` and `--mode replay`.

What is measured and what is assumed

I tag every number in these documents with one of four labels:

Label	Meaning
SOURCE	Public figure, source named
MEASURED	Produced by the evaluation script or the calculator in this repository

Label	Meaning
ASSUMED	My estimate. Each one has a date by which it gets checked (most in Phase 0, by 15 Nov 2026)
DECISION	A choice I made as the product owner

The 48 sample messages are **written by AI to look like real messages, not real parents' messages**. Results on them show that the design works end to end. They do not prove how it will do on real messages. The 100 real messages in Phase 0 do that.

Naming used in all documents

I name the people who take part instead of using a general word for them: **parent** (sends the inquiry), **front-office coordinator** (uses AdmitAssist every day), **principal** (handles sensitive cases), **school owner** (pays and approves school facts; in the pilot this is my father), **me** (product owner and vendor), **AI model provider** (rents the language model to me).